Hotel Loyalty Points Mall

Software Design Report

CS6451: Advanced Software Design • Autumn 2026–2027
University of Limerick

Team-Based Project

Lecturer: J. J. Collins
Submission: Week 8

Team Members

Fude Ding	26046091
Kewei Wu	26141825



Contents

Hotel Loyalty Points Mall 5

Project Report 5

1. Business Scenario 5

2. Executive Summary 5

Architecture diagrams . . . 5

3. Scope and Use Cases 8

Implementation status at a glance . . . 9

UC1 - Login and account information . . . 9

UC2 - Points transaction details . . . 9

UC3 - Redeem products . . . 10

UC4 - Manage points rules . . . 10

UC5 - Manually add points . . . 11

UC6 - Manage products . . . 11

4. Functional Architecture 11

Main packages . . . 12

5. Design Patterns 13

5.1 Layered architecture . . . 13

5.2 Strategy pattern for payment . . . 13

5.3 State pattern for orders . . . 14

5.4 Message-driven processing . . . 15

5.5 Optimistic concurrency . . . 15

5.6 Chain of Responsibility for post-login checks . . . 16

5.7 Template Method in login handlers . . . 16

5.8 Observer / publish-subscribe event handling . . . 16

5.9 Factory and Data Mapper patterns . . . 17

5.10 MVC and dependency injection . . . 17
5.11 Repository-level design principles . . . 18
6. Domain Model 19
7. Database Design 20
8. API and Page Endpoints 21
Authentication . . . 21
Guest pages and APIs . . . 21
Manager pages and APIs . . . 22
9. Security and Reliability 22
Authentication and authorization . . . 22
Data and transaction safety . . . 22
Asynchronous reliability . . . 22
Current limitations . . . 22
10. User Interface 23
11. Demo Data and Running the Application 23
Prerequisites . . . 23
Start the application . . . 23
Build and test . . . 23
Demonstration credentials . . . 24
Useful local tools . . . 24
12. Testing Status 24
13. UML and Supporting Documents 24
14. Implementation Gaps and Recommended Next Steps 25
Priority 1: [Key gap] Connect points rules to earning . . . 25
Priority 2: [Key gap] Add operational queue management . . . 25
Priority 3: [Partially complete] Make product availability explicit . . . 25

Priority 4: [Partially complete] Complete order lifecycle parity . . . 26

Test coverage status . . . 26

15. Conclusion 26

16. Technology Stack and Technical References 26

Technology and theory references . . . 27

17. Assignment Deliverables and Traceability Matrix 27

18. Software Lifecycle Discussion 28

19. Project Plan Summary 29

20. Quality Attributes and Quality Tactics 29

21. Transparency and Traceability 29

22. Critique of Design and Implementation 30

23. Reflection 30

Fordring . . . 30

Franklin . . . 30

24. Generative AI Use 31

25. References and Submission Checklist 31

Formal reference list . . . 31

Final export checklist . . . 31

Hotel Loyalty Points Mall

Project Report

> A hotel loyalty points management and redemption system built with Spring Boot, JSP, MyBatis, H2, Spring Security, and embedded Artemis JMS.

Project: CS6451 Advanced Software Design **Application:** hotel-loyalty **Version:** 0.0.1-SNAPSHOT **Java:** 21
Build: Maven **Last reviewed:** 27 October 2026

1. Business Scenario

Hotel guests earn and spend loyalty points. The hotel publishes redeemable products in a points mall, such as benefits or hotel-related rewards. A guest can pay with points alone, or with a mix of points and another payment method (currently cash or voucher).

Managers maintain the product catalogue and the rules that describe how points are earned. They can also manually award points to a guest. Manual awards go into a queue, so temporary processing failures do not lose the request.

The main business rules are:

1. A guest must be logged in and can only access their own resources.
1. A manager-only operation must not be available to a guest.
1. A redemption cannot use more points than the guest owns.
1. A redemption cannot use an unavailable product.
1. Remaining stock must never go negative.
1. A product with active orders cannot be deleted.
1. Every redemption and manual award creates a signed points transaction.

2. Executive Summary

Hotel Loyalty is a web application for managing a hotel's guest loyalty points and points-mall products. It has two authenticated user roles:

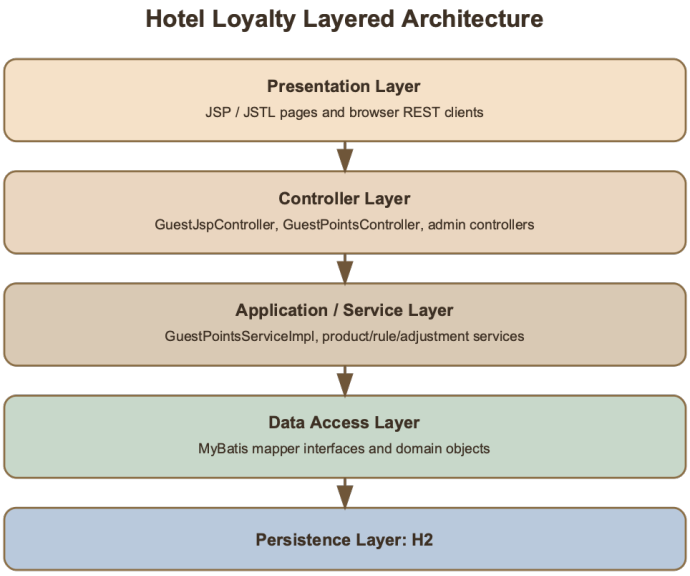
1. **Guest**: logs in, views the points mall, checks the current balance, picks a product, chooses a payment method, and redeems the product.
1. **Manager**: logs in through the manager portal, manages points-mall products, manages points rules, and manually adds points to a guest account.

The current implementation uses server-rendered JSP pages for the user interface and REST endpoints for browser-side administration and guest operations. Business logic lives in service classes, database access is isolated in MyBatis mappers, and Spring Security controls role-based access.

The implementation also shows several software-design techniques:

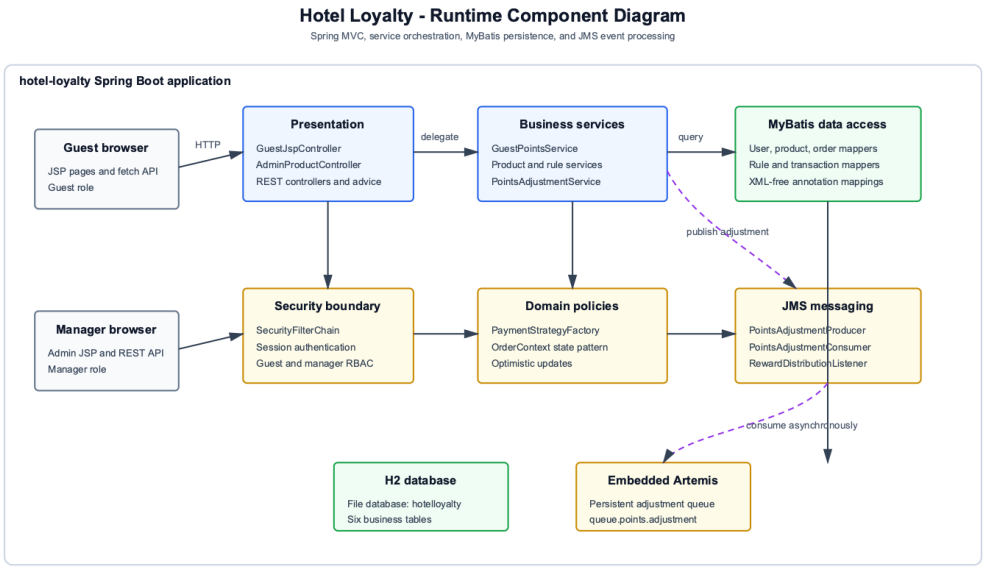
1. **Layered architecture**: controller, service, mapper, and persistence layers.
2. **Strategy pattern**: payment validation is selected through PaymentStrategyFactory.
3. **State pattern**: order lifecycle states are represented by the classes in the order package.
4. **Message-driven processing**: manual point adjustments are sent to an embedded Artemis queue and processed asynchronously.
5. **Optimistic concurrency control**: point balances and stock are updated with compare-and-update operations.

Architecture diagrams



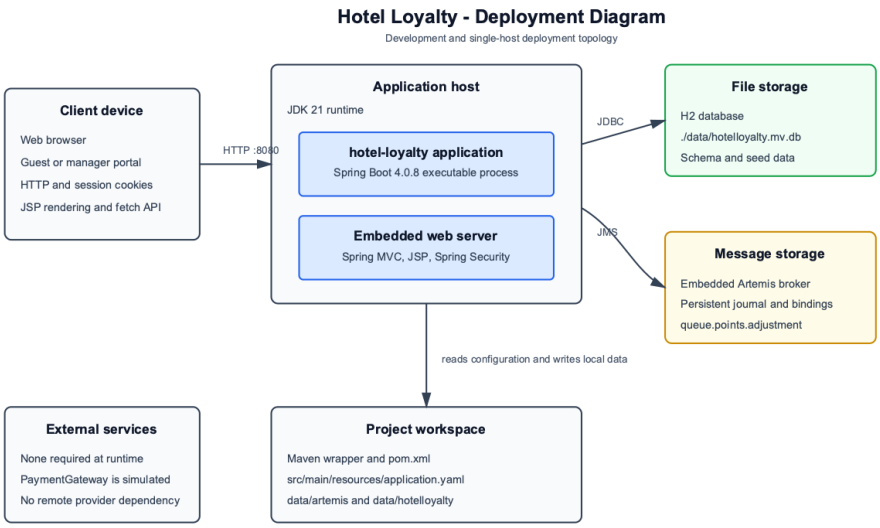
Layered architecture diagram

The design-time package boundaries are shown in package_diagram.svg. The runtime component collaboration is shown below.



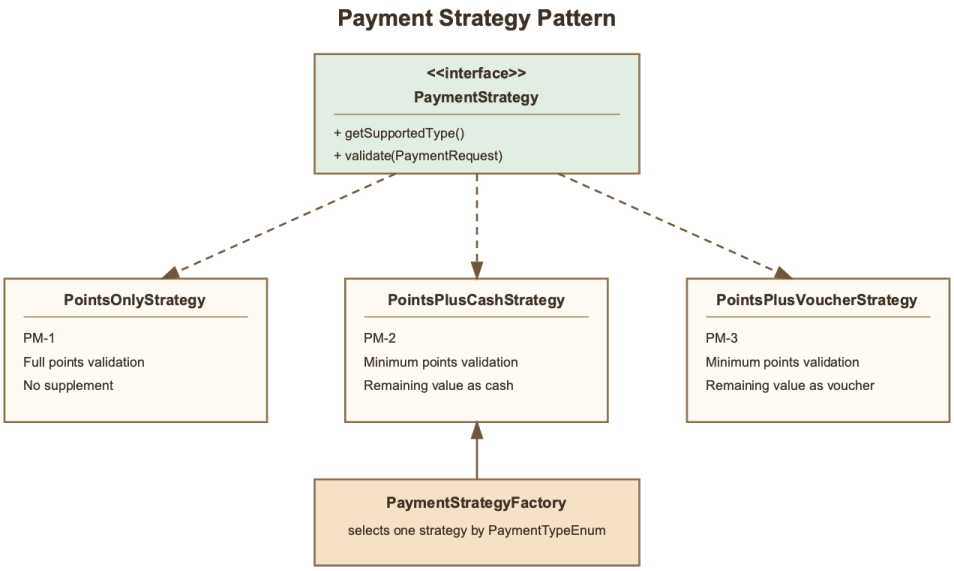
Component diagram

The single-host deployment topology is shown below.



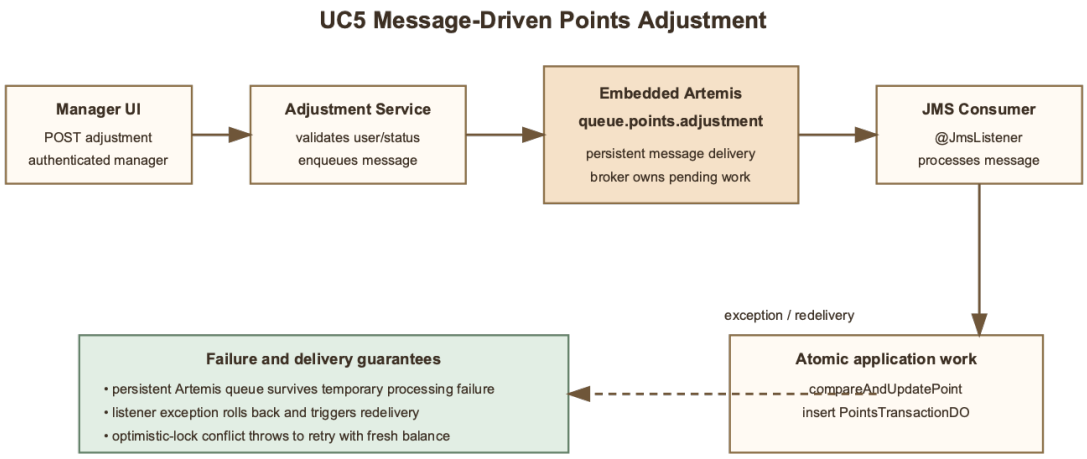
Deployment diagram

The payment design is a **Strategy pattern**, shown below. It is not a Chain of Responsibility: PaymentStrategyFactory picks exactly one strategy from PaymentTypeEnum, and strategies do not pass an unhandled request to one another. The controller and service call the chosen strategy directly.



Payment strategy UML diagram

The asynchronous points-adjustment flow and its reliability controls are shown below. The manager request is validated and placed on the persistent queue.points.adjustment Artemis queue. The listener updates the user balance and writes the transaction. An exception rolls back processing and causes redelivery; the optimistic compare-and-update also turns concurrent balance conflicts into retryable failures.

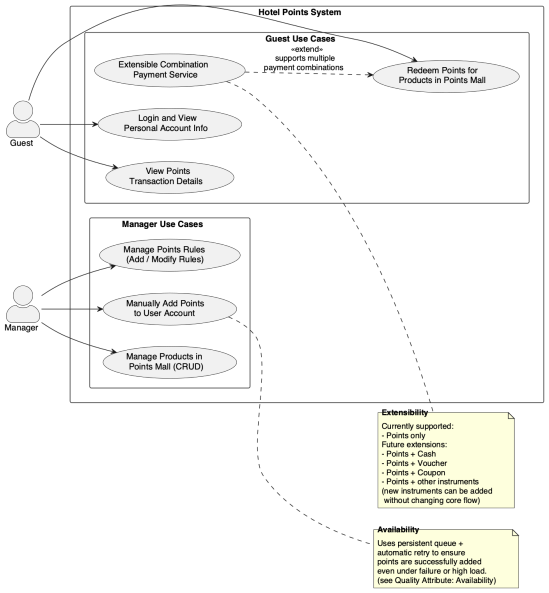


Message-driven points adjustment diagram

This report describes the implemented system, not the original conceptual model. Where a use case or UML artifact describes planned behavior that is not yet connected to the running code, it is explicitly marked as a gap.

3. Scope and Use Cases

The use-case catalogue has six primary use cases.



Use-case diagram

ID	Use case	Current implementation
UC1	Guest login and personal account information	**[Implemented]** Login, session identity, user ID, name, role, status, level, balance, registration time, and account page are implemented
UC2	View points transaction details	**[Implemented]** My Points shows signed history, type, related order, timestamp, and redemption orders

ID	Use case	Current implementation
UC3	Redeem points for products in the points mall	**[Implemented within simulated-payment scope]** Points, simulated cash, and simulated voucher flows validate and persist orders
UC4	Manager manages points rules	**[Partially implemented / key gap]** Rule CRUD works; **hotel-order points calculation and award processing are not connected**
UC5	Manager manually adds points	**[Core implemented / partially complete]** Persistent queue, consumer, retry, balance update, and EARN transaction work; **queue status, dead-letter handling, request status, and reason persistence are not implemented**
UC6	Manager manages products	**[Core implemented / partially complete]** Product CRUD and validation work; **guest-facing inventory availability needs better presentation**

Implementation status at a glance

These are the **key incomplete or not-yet-implemented features** and should not be described as fully delivered:

1. **UC1:** No known missing item within the current use-case scope. The account page shows every field currently stored for the user: ID, name, role, status, level, current points, and registration time.
1. **UC3:** No missing item within the agreed prototype scope. Cash and voucher are intentionally simulated; real provider integration is explicitly out of scope.
1. **UC4 — Rule execution:** managers can maintain rules, but no hotel-order earning event picks a rule and calculates/awards points.
1. **UC5 — Operational queue management:** there is no user-facing queued/processed/retrying/failed status, durable request ID, dead-letter administration, or persisted adjustment reason.
1. **UC6 — Availability presentation:** the backend filters available products, but the guest-facing product view does not show a complete stock/availability state for every product.
1. **Order lifecycle orchestration:** the redemption service uses the order state classes for the pay transition, but cancellation, fulfilment, and shipment workflows are not implemented.

UC1 - Login and account information

The application has separate login pages:

1. /login for guests
1. /admin/login for managers

Authentication uses Spring Security sessions and BCrypt passwords. The guest points page shows the guest name, level, balance, registration time, and redemption records. The account page shows all stored account fields: user ID, name, role, status, membership level, current points, and registration time. **UC1 is complete for the current data model and viewing scope.**

UC2 - Points transaction details

The database and mapper support transaction history through PointsTransactionMapper.selectByUid. Redemption and manual-award flows also create PointsTransactionDO records. MyPointsVO and guest_my_points.jsp show the complete signed transaction history in the Points History tab:

1. transaction date;
1. positive or negative points change;
1. transaction type (EARN or REDEEM);

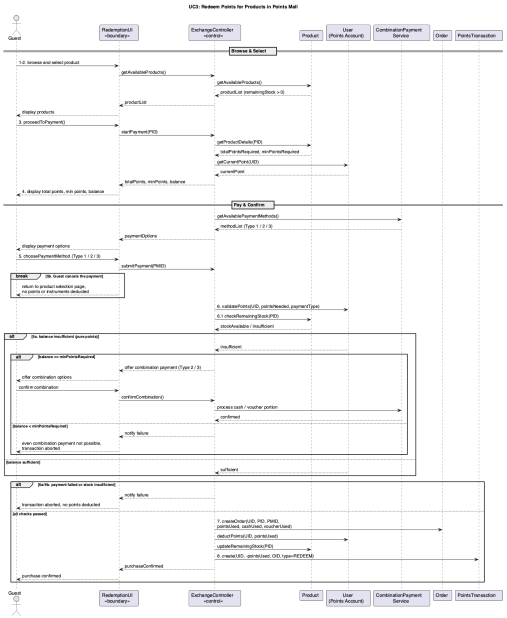
1. related order ID, when applicable.

UC3 - Redeem products

The guest redemption flow is implemented by GuestPointsServiceImpl:

1. Load and validate the guest and product (including an in-stock pre-check).
2. Validate the chosen payment strategy.
3. Create a pending order.
4. Atomically deduct points with an optimistic balance check (compare-and-update).
5. Settle the non-points portion through PaymentGateway (simulated cash charge or voucher redemption).
6. Mark the order as paid through the OrderContext state pattern with a compare-and-update status transition.
7. Insert a negative points transaction linked to the order.
8. Publish a PointsDeductedEvent; RewardDistributionListener runs synchronously inside the same transaction and deducts one unit of product stock as the reward-distribution step.

The My Redemptions tab reads the same paid order records through OrderMapper.selectByUid, so points-only, cash, and voucher redemptions all appear after successful completion.



Redemption sequence diagram

The supported payment strategies are:

1. points only;
1. points plus cash;
1. points plus voucher.

The cash and voucher interactions are intentionally simulated for this academic prototype. Real external payment authorization, capture, refund, timeout reconciliation, and provider-failure handling are **out of scope**, not incomplete implementation work.

UC4 - Manage points rules

Managers can list, create, and update rules. The service validates rule type and the time range for period rules. Level rules clear the threshold because they do not use a minimum external order amount.

The rule data is stored and managed, but no external hotel-order earning process in this application reads the rules to calculate newly earned points. The management part of UC4 is therefore implemented; **the following calculation and award integration is not implemented**.

UC5 - Manually add points

Managers verify a target user, enter a positive point amount, and may optionally provide a reason. The request is sent to the persistent Artemis queue. PointsAdjustmentConsumer updates the balance and creates an EARN transaction. Exceptions cause message redelivery, which gives automatic retry behavior.

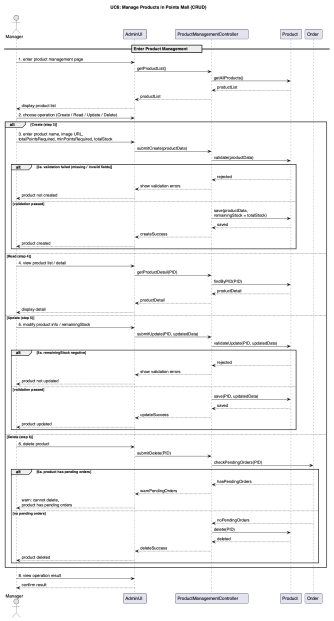
The optional reason is currently written to the application log, not persisted in points_transaction, whose schema has no reason column. **There is also no queue-status API, durable request ID, dead-letter administration, or user-facing retry status.**

UC6 - Manage products

Managers can:

1. create a product;
1. list products;
1. get product details;
1. update product information and stock;
1. delete a product when it has no active orders.

On creation, remainingStock is set to totalStock. The service validates that minimum points do not exceed total points and that remaining stock does not exceed total stock. **The guest-facing catalogue currently filters unavailable products instead of showing a complete availability status.**



Product management sequence diagram

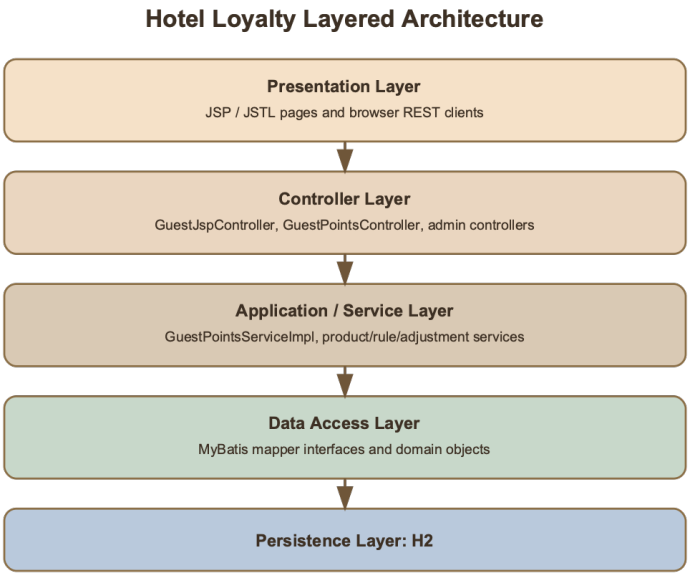
4. Functional Architecture

The application follows a layered web architecture:

```
JSP pages / REST clients
          |
          v
Controllers
          |
          v
Application services
          |
          v
MyBatis mappers
          |
          v
H2 database
```

The asynchronous manual-award path is:

```
Manager UI
    |
    v
PointsAdjustmentController
    |
    v
PointsAdjustmentService
    |
    v
Artemis queue
    |
    v
PointsAdjustmentConsumer
    |
    +--> UserMapper
    |
    +--> PointsTransactionMapper
```



Layered architecture diagram

Main packages

Package	Responsibility
controller	REST endpoints and JSP page controllers
service	Application service interfaces
service.impl	Business rules and transaction orchestration
domain	Persistence/domain objects
dto	Validated request payloads
vo	View and API response objects
mapper	MyBatis data-access interfaces
payment	Payment strategy abstractions and implementations
messaging	Artemis producer, consumer, messages, and events
order	Order state abstraction
security	Authentication, login flow, and role handling
config	Spring Security and JMS configuration

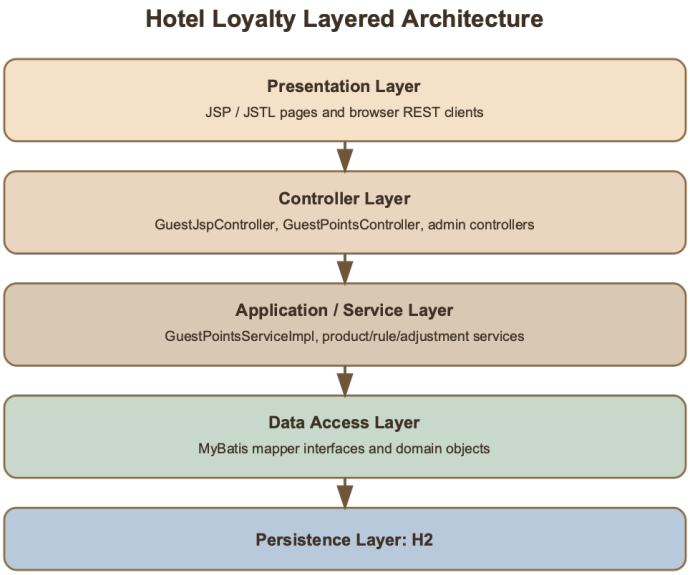
5. Design Patterns

5.1 Layered architecture

Controllers do not directly implement persistence logic. For example:

```
ProductManagementController
        -> ProductManagementService
        -> ProductMapper
        -> ProductDO
```

This keeps HTTP concerns separate from business validation and database access.



Layered package architecture diagram

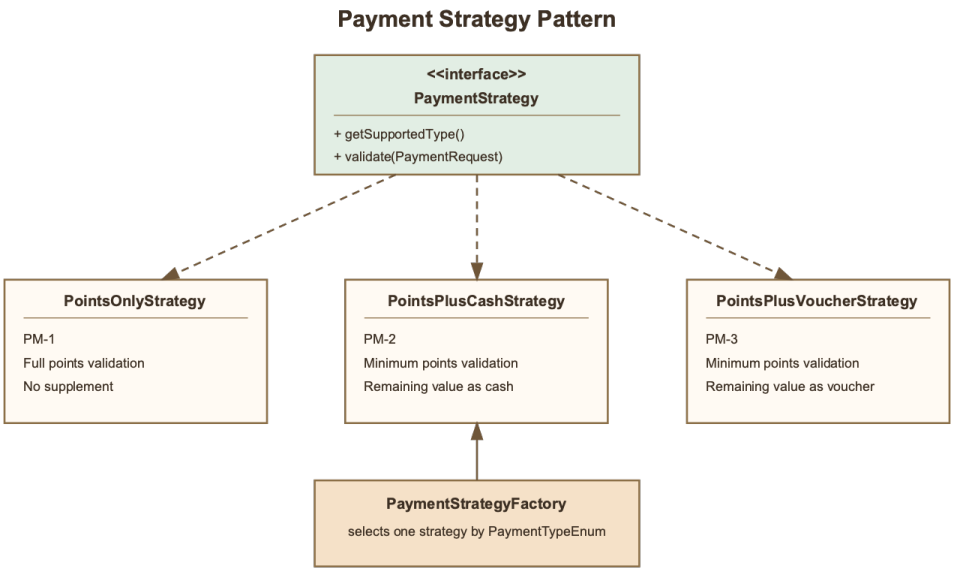
5.2 Strategy pattern for payment

PaymentStrategy defines the validation contract. The concrete strategies are:

1. PointsOnlyStrategy
1. PointsPlusCashStrategy
1. PointsPlusVoucherStrategy

PaymentStrategyFactory picks a strategy by PaymentTypeEnum. A new payment instrument can therefore be added as another strategy without changing the main redemption orchestration.

The class relationship and concrete implementations are shown above. This is not a Chain of Responsibility: there is no sequential handler chain, fallback handler, or next reference.



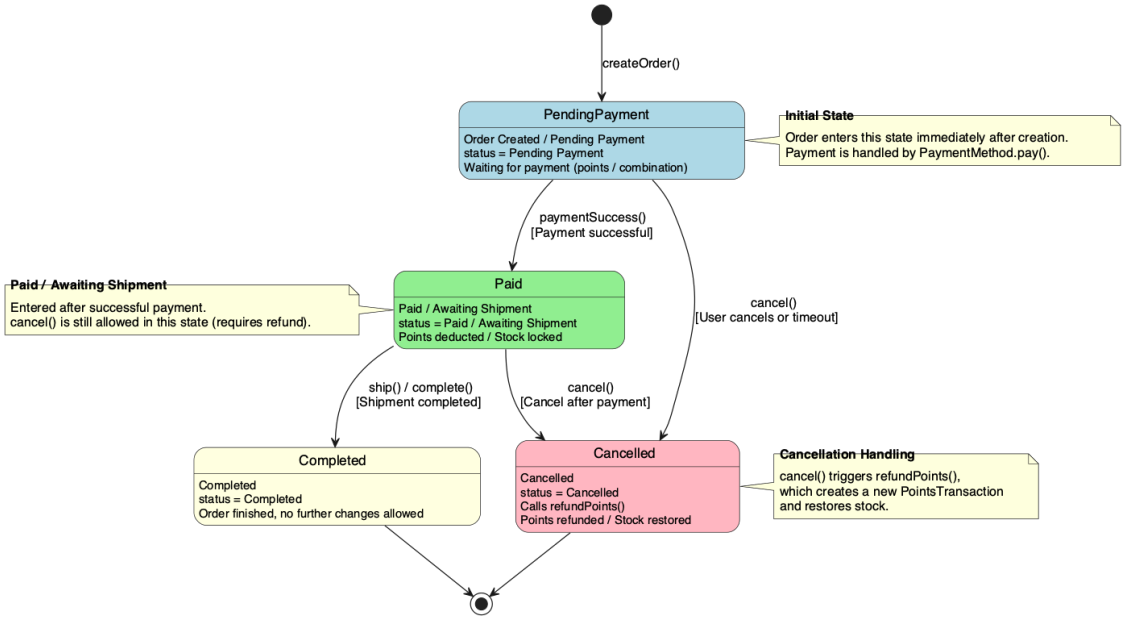
Payment strategy UML diagram

5.3 State pattern for orders

The order package contains:

1. OrderState
1. PendingPaymentState
1. PaidState
1. CompletedState
1. CancelledState
1. OrderContext

The current guest redemption service uses OrderContext to validate the pay transition, then persists the new status with a compare-and-update. The state classes provide the model for order lifecycle behavior and can be expanded as cancellation and fulfilment workflows are added.

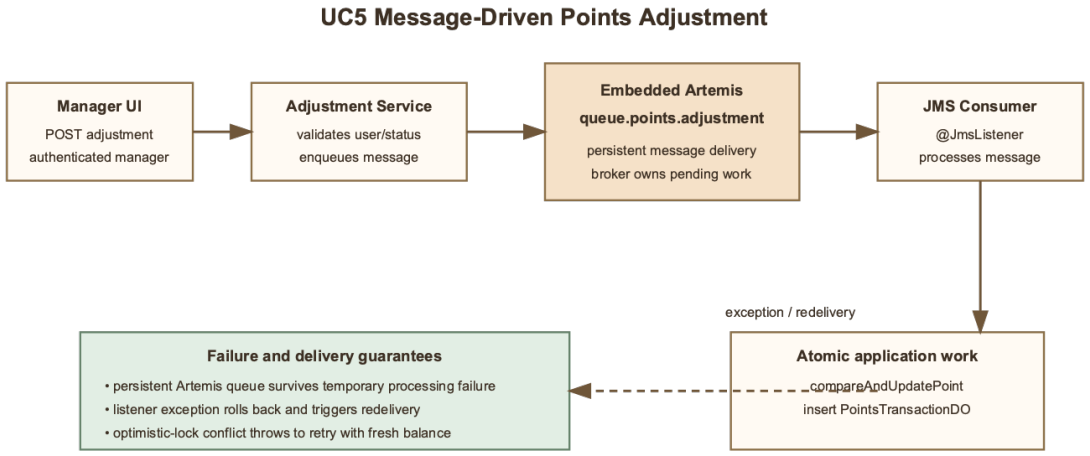


Order state diagram

5.4 Message-driven processing

Manual point awards are decoupled from the manager request through Artemis JMS. The producer acknowledges the request by enqueueing it, while the consumer performs the balance update and transaction insert. An exception lets the broker redeliver the message.

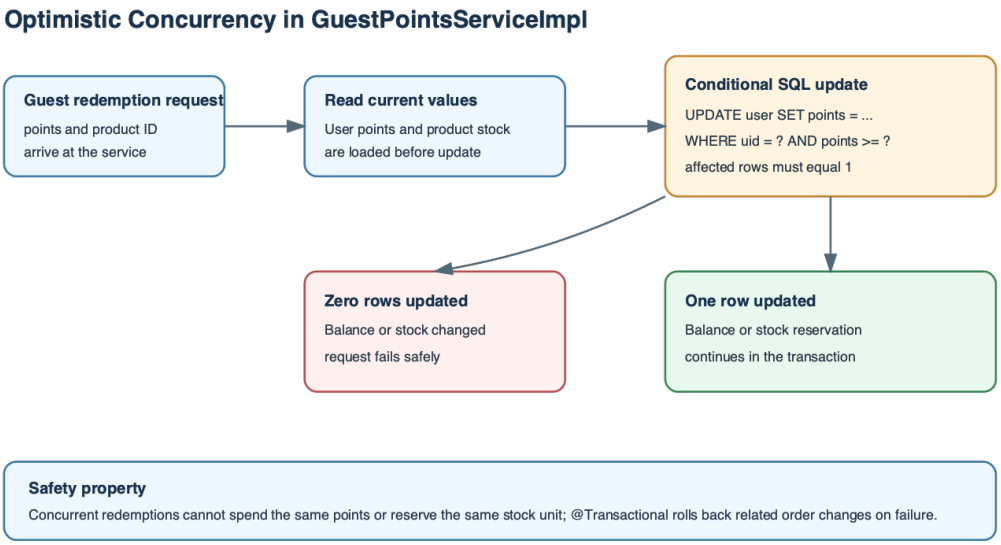
The queue, producer, consumer, database writes, and retry path are shown above. Persistent delivery, listener rollback/redelivery, and optimistic locking provide the implementation's reliability mechanism; there is no claim of exactly-once processing beyond the current transaction and message configuration.



Message-driven processing diagram

5.5 Optimistic concurrency

Point and stock updates use conditional database updates. A redemption fails if the stored balance or stock changed between the initial read and the update. This stops two concurrent requests from spending the same points or stock unit.



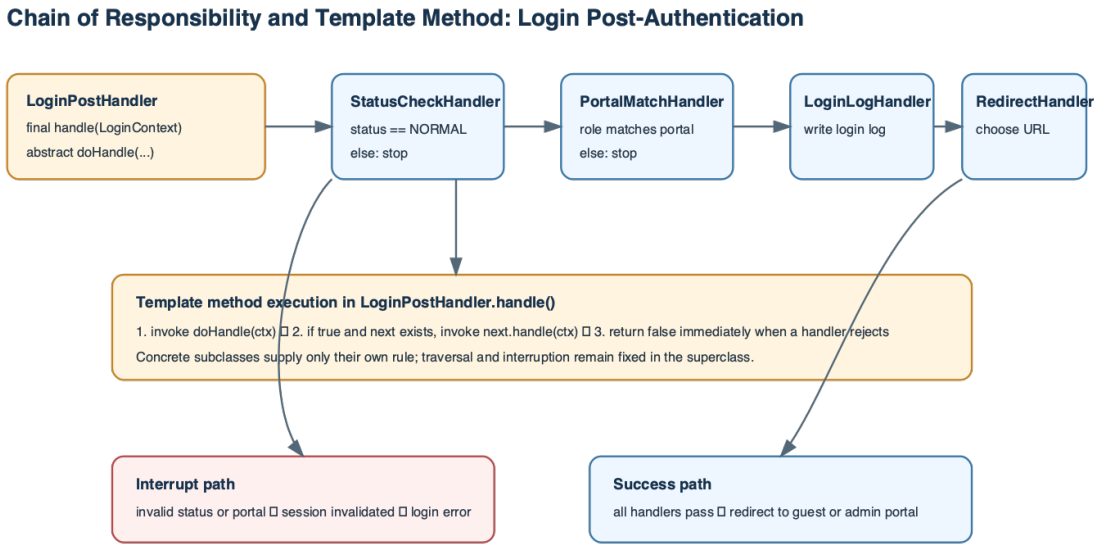
Optimistic concurrency diagram

5.6 Chain of Responsibility for post-login checks

The security package contains a hand-written Chain of Responsibility for business checks that run after Spring Security has authenticated the password:

```
StatusCheckHandler
        -> PortalMatchHandler
        -> LoginLogHandler
        -> RedirectHandler
```

Each LoginPostHandler performs one check and passes the shared LoginContext to its successor. Returning false stops the chain. This is separate from Spring Security's framework SecurityFilterChain; the project-owned chain handles application-specific status, portal, logging, and redirect decisions.



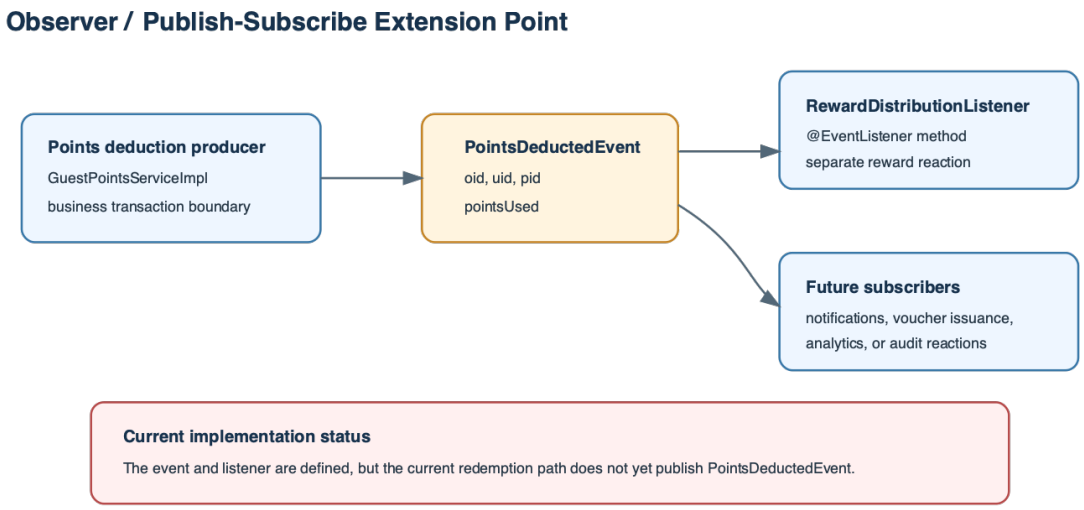
Login Chain of Responsibility and Template Method

5.7 Template Method in login handlers

LoginPostHandler.handle() is a final template method. It runs the common control flow (run the current handler, then call the successor when the check passes), while concrete handlers implement the variable doHandle() step. This keeps chain traversal consistent and limits each handler to one business rule.

5.8 Observer / publish-subscribe event handling

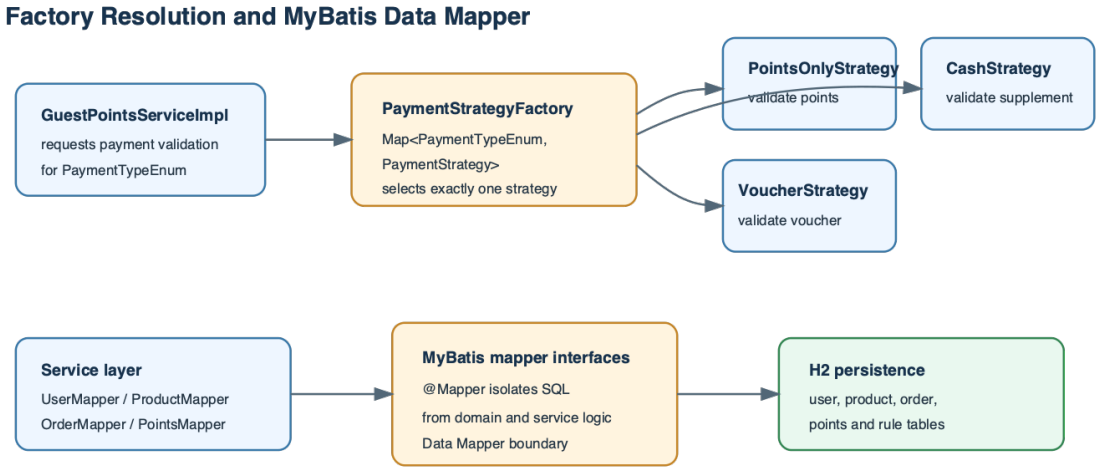
RewardDistributionListener and PointsDeductedEvent provide an active application-level Observer / publish-subscribe flow. When GuestPointsServiceImpl.redeem finishes deducting points, settling payment, and recording the order, it publishes a PointsDeductedEvent. RewardDistributionListener observes the event and deducts one unit of product stock as the reward-distribution step, keeping reward distribution separate from point deduction. Spring's synchronous @EventListener runs the listener inside the publisher's transaction, so the stock deduction joins the same redemption transaction — atomicity is preserved while the logic stays decoupled (Open/Closed Principle). More listeners, such as notification or voucher issuance, can subscribe to the same event without touching the redemption service or this listener.



Observer and event publication diagram

5.9 Factory and Data Mapper patterns

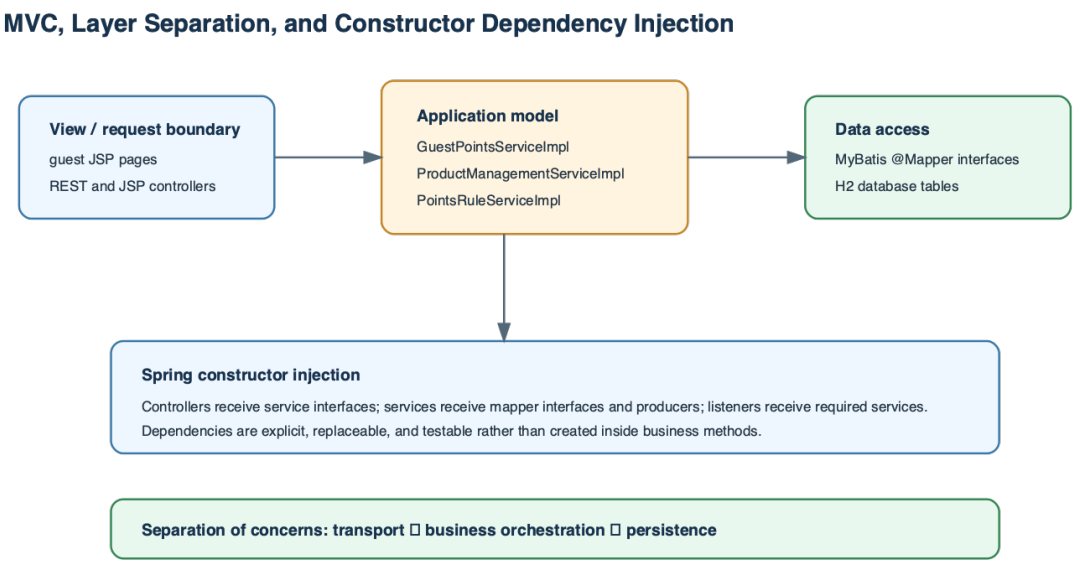
PaymentStrategyFactory is a Factory object that resolves a payment strategy from PaymentTypeEnum. The MyBatis mapper interfaces (UserMapper, ProductMapper, OrderMapper, and others) implement the Data Mapper pattern: persistence operations are kept outside the domain objects and services.



Factory and Data Mapper diagram

5.10 MVC and dependency injection

The JSP controllers and REST controllers provide the Model-View-Controller separation: controllers handle requests, services provide application behavior, and JSP pages or Result objects provide the view/response. Constructor injection is used throughout the controllers, services, producers, consumers, and listeners, making dependencies explicit and testable.



MVC and dependency injection diagram

5.11 Repository-level design principles

The implementation applies these principles:

1 **Single Responsibility Principle:** controllers handle transport, services orchestrate business rules, mappers handle persistence, and payment strategies validate one payment type.

1 **Open/Closed Principle:** new payment strategies and event listeners can be added without rewriting the redemption orchestration or existing listeners.

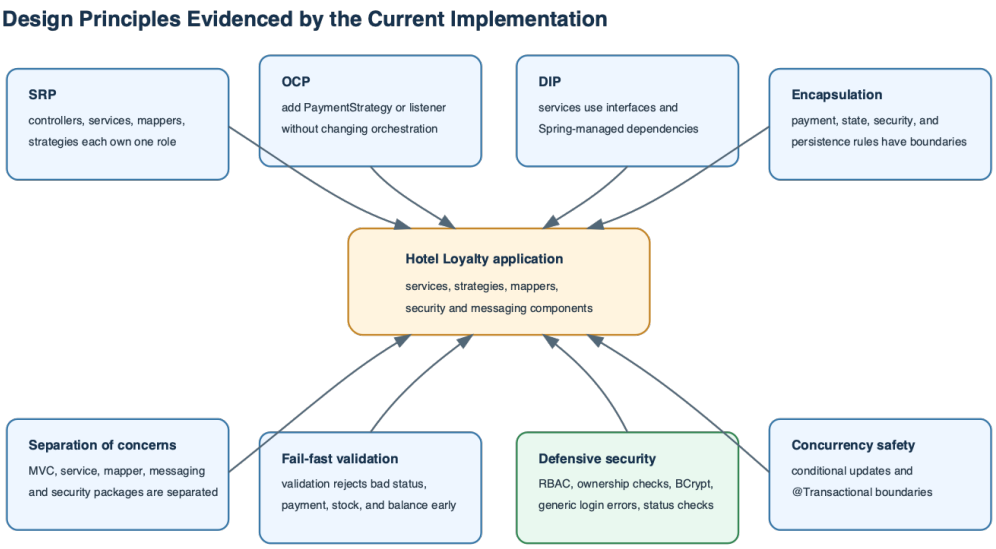
1 **Dependency Inversion Principle:** services depend on mapper and producer interfaces or abstractions rather than constructing database or messaging infrastructure directly.

1 **Encapsulation:** payment validation, order transitions, security checks, and persistence operations are kept behind focused interfaces and service boundaries.

1 **Separation of concerns:** authentication, authorization, business validation, persistence, messaging, and presentation are placed in separate packages and layers.

1 **Fail-fast validation:** invalid status, portal, payment, stock, balance, and input conditions are rejected before dependent operations continue.

1 **Defensive security design:** role-based URL rules, ownership checks, BCrypt password hashing, generic login errors, and explicit account-status checks reduce unauthorized access and information disclosure.



Design principles evidence diagram

6. Domain Model

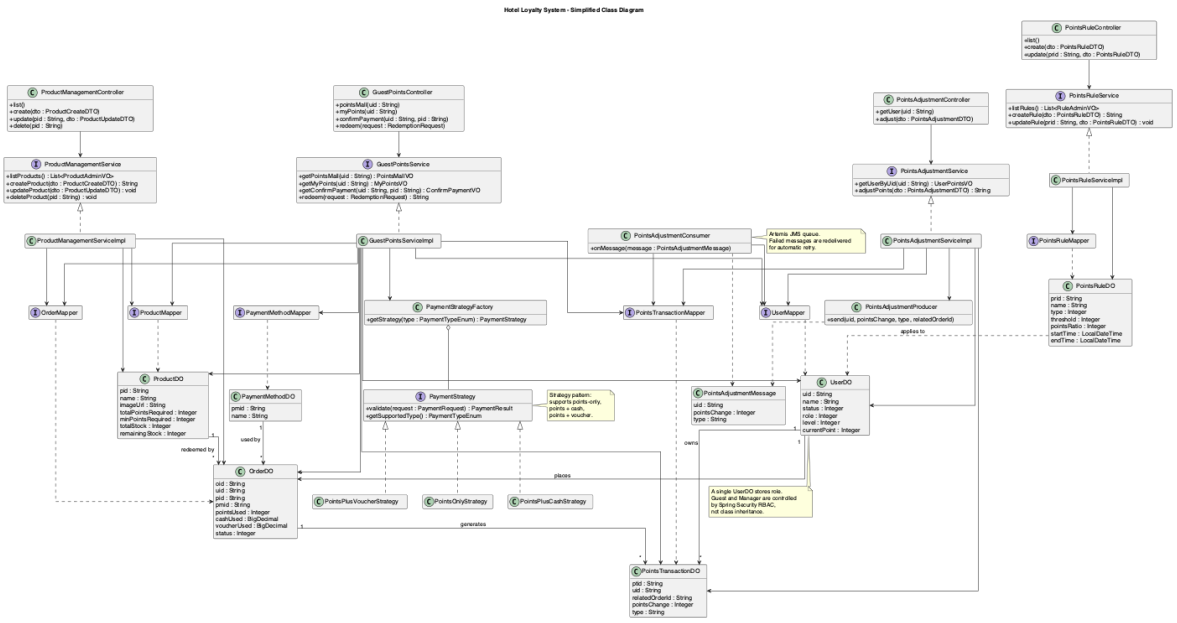
The core persistent entities are:

Entity	Purpose
UserDO	Guest or manager identity, role, status, level, and point balance
ProductDO	Points-mall product and stock information
PaymentMethodDO	Configured payment method catalogue
OrderDO	A guest redemption request and its payment data
PointsTransactionDO	Signed points audit record
PointsRuleDO	Manager-maintained earning rule definition

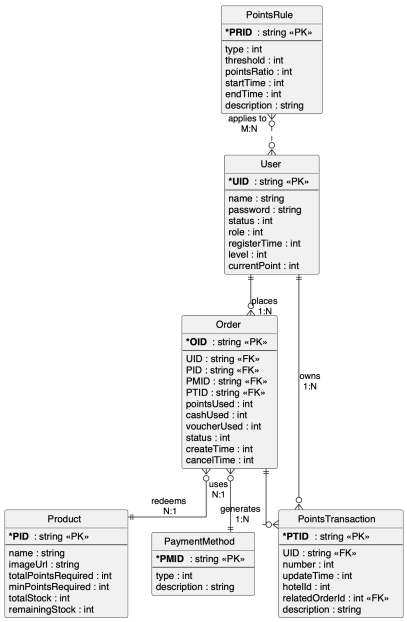
The current implementation stores both guest and manager accounts in UserDO. They are not Java subclasses. Spring Security maps the stored role to ROLE_GUEST or ROLE_MANAGER.

The main relationships are:

```
User 1 ---- * Order
User 1 ---- * PointsTransaction
Product 1 ---- * Order
PaymentMethod 1 ---- * Order
Order 1 ---- * PointsTransaction
PointsRule ..> User
```



Class diagram



Entry relationship diagram

The application uses business identifiers (uid, pid, pmid, oid, ptid, and prid) in service and mapper operations. The database schema intentionally keeps foreign-key enforcement in the application layer.

7. Database Design

The database is H2 in file mode:

```
./data/hotelloyalty
```

Schema initialization is enabled on startup. The main tables are:

```
user
product
payment_method
order
points_transaction
points_rule
```

Important data rules:

1. all tables have a generated numeric id;
1. business identifiers have unique constraints;
1. points_change is signed;
1. remaining_stock is maintained separately from total_stock;
1. related_order_id is nullable for non-order earnings;
1. orders and points transactions keep creation and modification timestamps.

The schema is defined in schema.sql, and initial demonstration data is in data.sql.

8. API and Page Endpoints

Authentication

Method	Endpoint	Purpose
GET	/login	Guest login page
GET	/admin/login	Manager login page
POST	/auth/login	Spring Security login processing
POST	/logout	Session logout

Guest pages and APIs

Method	Endpoint	Purpose
GET	/guest/{uid}/account	Render the guest account page
GET	/guest/{uid}/points-mall	Render the guest points mall
GET	/guest/{uid}/my-points	Render balance and redemption page
GET	/guest/{uid}/confirm-payment/{pid}	Render payment confirmation
POST	/guest/{uid}/redemptions	Submit JSP redemption
GET	/api/guest/{uid}/account	Get account data
GET	/api/guest/{uid}/points-mall	Get mall data
GET	/api/guest/{uid}/my-points	Get guest points data
GET	/api/guest/{uid}/confirm-payment/{pid}	Get payment data
POST	/api/guest/redemptions	Submit REST redemption

Guest endpoints verify that the URL or request UID matches the logged-in session identity.

Manager pages and APIs

Method	Endpoint	Purpose
GET	/admin/products	Product management page
GET	/admin/rules	Points-rule management page
GET	/admin/points	Manual points adjustment page
GET	/api/admin/products	List products
POST	/api/admin/products	Create product
GET	/api/admin/products/{pid}	Get product
PUT	/api/admin/products/{pid}	Update product
DELETE	/api/admin/products/{pid}	Delete product
GET	/api/admin/rules	List rules
POST	/api/admin/rules	Create rule
GET	/api/admin/rules/{prid}	Get rule
PUT	/api/admin/rules/{prid}	Update rule
GET	/api/admin/points/user/{uid}	Verify target user
POST	/api/admin/points/adjustment	Enqueue manual point adjustment

All /admin/ **and /api/admin/** routes require the manager role.

9. Security and Reliability

Authentication and authorization

1. Session-based Spring Security authentication.
1. BCrypt password encoding.
1. Separate guest and manager login portals.
1. URL-level role-based access control.
1. Guest ownership checks against SecurityContext.
1. Explicit 30-minute session timeout.

Data and transaction safety

1. Redemption is transactional.
1. Point balance updates use compare-and-update semantics.
1. Stock deduction checks remaining stock in the update operation.
1. Product deletion checks for active orders before deletion.
1. Validation annotations are used on request DTOs.

Asynchronous reliability

1. Embedded Artemis is configured as persistent.
1. Manual adjustments are queued before processing.
1. Consumer exceptions trigger broker redelivery.
1. Point adjustment and transaction creation are performed by the consumer.

Current limitations

1. CSRF is disabled because the current JSP JavaScript calls REST endpoints without CSRF tokens. This is acceptable for the academic prototype but should be fixed for production.
1. No user-facing queue status or dead-letter workflow is exposed.
1. The adjustment reason is logged but not stored in the database.
1. The points-rule table is not yet connected to an external hotel-order earning flow.

10. User Interface

The UI is server-rendered using JSP and JSTL.

View	Purpose
login.jsp	Guest login
admin_login.jsp	Manager login
guest_account.jsp	Show account information
guest_points_mall.jsp	Browse products
guest_confirm_payment.jsp	Select and confirm payment
guest_my_points.jsp	Show balance and redemption records
admin_products.jsp	Product CRUD
admin_points_rules.jsp	Points rule CRUD
admin_points_adjust.jsp	Manual point award
products.jsp	Product-list demonstration page

The working demonstration page is:

```
http://localhost:8080/demo/products
```

The manager product page is:

```
http://localhost:8080/admin/products
```

11. Demo Data and Running the Application

Prerequisites

1. JDK 21 or a compatible newer JDK
1. Maven Wrapper included in the project

Start the application

From the application directory:

```
cd Project/hotel-loyalty
./mvnw spring-boot:run
```

On Windows:

```
cd Project\hotel-loyalty
mvnw.cmd spring-boot:run
```

The default server address is:

```
http://localhost:8080
```

Build and test

```
cd Project/hotel-loyalty
./mvnw test
./mvnw package
```

The generated executable JAR is placed under Project/hotel-loyalty/target/.

Demonstration credentials

The seed data defines:

Role	Username	Password
Manager	admin	admin123
Guest	guest	guest123

These credentials are for local demonstration only and must be replaced in a deployed environment.

Useful local tools

1. H2 Console: http://localhost:8080/h2-console
1. OpenAPI JSON: http://localhost:8080/v3/api-docs
1. Swagger UI: http://localhost:8080/swagger-ui.html

For the H2 console, use the JDBC URL configured in application.yaml and the local sa user.

12. Testing Status

The repository currently contains a Spring context smoke test:

```
src/test/java/com/hotel/hotel_loyalty/HotelLoyaltyApplicationTests.java
```

The current automated suite has 103 tests covering:

1. Login, account-status, guest-ownership, and manager-role security behavior.
2. Product create, update, delete, validation, and active-order protection.
3. Points-only, points-plus-cash, and points-plus-voucher validation.
4. Minimum-point, supplement-conversion, insufficient-balance, and out-of-stock redemption.
5. Atomic balance, stock, order, and points-transaction changes.
6. Voucher redemption transaction-history linkage.
7. Manual-award queue processing and optimistic-lock retry behavior.
8. Event-driven reward distribution: PointsDeductedEvent publication and listener stock deduction.
9. Mapper, controller, service, state, and application-context behavior.

The suite does not yet provide browser-level end-to-end coverage for every use case, real external payment-provider tests, points-rule calculation integration, or queue dead-letter operations.

The full suite currently passes with ./mvnw clean test.

13. UML and Supporting Documents

The repository contains these supporting artifacts:

Artifact	Location
Use-case catalogue	Use_case_1.0.xlsx
Detailed UC3 and UC6 descriptions	Use_Case_Description_UC3_UC6_v1.1.xlsx
Original class diagram	class_diagram_1.0.svg
Use-case diagram	all_user_case.svg
Product-management sequence diagram	manage_products.svg

Artifact	Location
Redemption sequence diagram	redeem_points_for_products.svg
Order state diagram	order_state_diagram_1.0.svg
Entry relationship diagram	entry_relationship_diagram.svg
Layered package architecture	layered_architecture.svg
Payment strategy UML	payment_strategy_uml.svg
Message-driven points adjustment	message_driven_points_adjustment.svg
Optimistic concurrency control	optimistic_concurrency.svg
Login Chain of Responsibility and Template Method	login_chain_template.svg
Observer event extension point	observer_event.svg
Factory and Data Mapper	factory_data_mapper.svg
MVC and dependency injection	mvc_dependency_injection.svg
Design principles evidence	design_principles.svg
Database schema	schema.sql

The original UML assets describe the conceptual design and should be read together with the current implementation notes in this report. In particular:

1. the running code uses one UserDO with role-based security rather than Java Guest and Manager subclasses;
1. payment methods are stored as database records while payment validation is implemented through PaymentStrategy;
1. the current guest page shows both redemption records and the full points transaction history;
1. order-state classes exist; the redemption service uses them to validate the pay transition, and cancellation/fulfilment workflows remain to be connected.

14. Implementation Gaps and Recommended Next Steps

> Reviewed against the current code on 27 October 2026. All four gaps below are still open.

Priority 1: [Key gap] Connect points rules to earning

PointsRuleServiceImpl only performs CRUD on rules, and PointsRuleMapper.selectByTypeAndThreshold is not called by any main-code flow. Introduce an external-order earning service that picks an applicable PointsRuleDO, calculates the points from the ratio, and sends the resulting earning event through the existing adjustment/message mechanism.

Priority 2: [Key gap] Add operational queue management

PointsAdjustmentMessage carries no reason and no durable request ID, points_transaction has no reason column, the reason is only written to the application log, and JmsConfig defines no dead-letter queue. Persist the manual-adjustment reason, add a durable request identifier, and expose queued, processed, retrying, and permanently failed states with dead-letter administration.

Priority 3: [Partially complete] Make product availability explicit

ProductVO only exposes pid, name, imageUrl, and totalPointsRequired; getPointsMall filters products through selectAvailable(). The backend filters out-of-stock products, but the guest-facing model should expose availability and stock status explicitly where the use case requires it.

Priority 4: [Partially complete] Complete order lifecycle parity

The redemption service now validates the pay transition through OrderContext and persists it with a compare-and-update, but OrderContext.cancel() and complete() are not called by any workflow. Connect the existing order-state classes to cancellation, fulfilment, and shipment workflows. UC1 account viewing is complete; the remaining gap is order lifecycle behavior, not account data.

Test coverage status

The current automated suite contains 103 tests, all passing with ./mvnw clean test; the covered areas are listed in section 12.

15. Conclusion

The current project is a working academic prototype of a hotel loyalty points mall. Its strongest implemented areas are the core redemption transaction, role-based manager/guest separation, product CRUD, strategy-based payment validation, and asynchronous manual point adjustment.

The main differences between the original design documents and the current code are implementation-level rather than architectural: the application uses role-based persistence instead of user inheritance, strategy classes instead of payment-method subclasses, and service/mapper layers instead of domain objects containing all behavior.

The next meaningful step is to show product availability more explicitly and connect managed points rules to an actual points-earning workflow. Once those are implemented and covered by browser-level tests, the project will have much stronger traceability from the use-case catalogue through the UML artifacts to the running application.

16. Technology Stack and Technical References

This section identifies the technologies used by the implemented system and the role of each technology in the solution.

Area	Technology	Use in this project
Language/runtime	Java 21	Application source, domain objects, services, tests, and build target
Application framework	Spring Boot 4.0.8	Application bootstrapping, auto-configuration, dependency management, and embedded server
Web layer	Spring MVC / WebMVC	REST controllers, page controllers, request binding, validation, and exception handling
Presentation	JSP, JSTL, HTML, CSS, JavaScript	Server-rendered guest and manager pages and browser-side API calls
Security	Spring Security, BCrypt	Session authentication, guest/manager role-based authorization, password hashing, and ownership checks
Persistence	MyBatis 4.0.1	Data Mapper interfaces and SQL-backed access to the application tables
Database	H2 Database	Local file-mode relational database for development, demonstration, and automated tests
Messaging	Jakarta Messaging (JMS), embedded Apache ActiveMQ Artemis	Persistent queue and asynchronous processing of manager point adjustments
API documentation	Springdoc OpenAPI 3.1.1, Swagger UI	OpenAPI 3 document and interactive API documentation
Validation	Jakarta Bean Validation / Spring Boot validation	DTO constraints and fail-fast request validation

Area	Technology	Use in this project
Build	Apache Maven and Maven Wrapper	Reproducible dependency resolution, compilation, testing, and packaging
Testing	JUnit 5, Spring Boot test support, Spring Security Test, MyBatis test support	Unit, integration, controller, security, service, mapper, and context tests
Code generation	Lombok	Reduced boilerplate for selected Java model and service classes
Modelling	UML/SVG, Excel use-case and candidate-object workbooks	Requirements, class, sequence, state, component, deployment, package, and data-model communication
Version control	Git and GitHub	Source history, collaboration, traceability, and version-control evidence

The implementation is intentionally a single deployable Spring Boot application. H2 and embedded Artemis keep the academic prototype self-contained; they are replaceable infrastructure choices rather than business-domain dependencies.

Technology and theory references

The implementation and design discussion are grounded in the five references listed in Section 25. The correspondence is:

1. **Architecture and quality attributes:** Bass, Clements, and Kazman.
1. **Strategy, State, Observer, Factory, and Chain of Responsibility:** Gamma et al.
1. **MVC, Data Mapper, and transaction-script/service-layer separation:** Fowler.
1. **Spring MVC, dependency injection, Spring Security, and Spring Boot:** Walls.
1. **Agile lifecycle, principles, and the complexity/agility trade-off:** Martin.

The online framework documentation used for implementation verification is linked in the technology table where appropriate; it is not counted as part of the formal five-item reference list.

17. Assignment Deliverables and Traceability Matrix

The following matrix converts the supplied assignment workbook into a report-ready submission checklist. “Evidence” links to the current repository artifact; “status” is the state of the evidence in this repository, not a claim that every optional extension has been implemented.

Assignment item	Required report output	Evidence in this repository	Status
3.1 Business scenario	Maximum one-page narrative	Section 1; 3.1 business scenario%20narrative%20description%20of%20your%20business%20scenario.docx)	Complete
3.2 Software lifecycle	Maximum two-page lifecycle discussion and complexity/agility tension	3.2 lifecycle discussion; Section 18	Complete
3.3 Project plan	Scope, ownership, schedule, dependencies, and completion evidence	Project plan; Section 19	Evidence available; narrative summary added
4.1 Requirements	Use-case diagram and two detailed key use cases	Use-case diagram; UC3 and UC6 descriptions; Section 3	Complete
4.2 Quality attributes	At least one “-ility” must be extensibility, plus quality tactics	Section 20; Sections 5 and 9	Complete
4.3 GUI prototypes	Two GUI prototypes and explanation of user flow	GUI prototypes; Section 10	Complete

Assignment item	Required report output	Evidence in this repository	Status
5.0 Architecture	Maximum two-page architecture discussion, two architectural patterns, and a high-level package view	Architecture discussion; Sections 4 and 5; package diagram	Complete
6.1–6.5 Design sketches	Candidate objects, class, sequence/communication, state, and ER diagrams	Candidate objects; UML assets; Section 13	Complete
7.1–7.4 Transparency	Package/class counts, authorship and LOC, team contribution, patterns/testing/CI-CD ownership	Section 21; source tree; GitHub history	Counts/LOC table must be refreshed before final submission
8.1–8.4 Code evidence	MVC/REST, four patterns, key use cases, and automated testing snippets	Sections 5, 8, and 12; src/main/java; src/test/java	Complete
8.5 Version control	Visual evidence of version-control use	Git repository: <https://github.com/fordringd2008/CS6451-Advanced-software-design>	Complete; add screenshot if required by marking rubric
9 Added value	Additional design or implementation value	Sections 5.4–5.10, 9, and 20	Complete
10 Recovered architecture	Design-time package, component, and deployment blueprints	package diagram, component diagram, deployment diagram	Complete
11 Critique	Maximum one-page critique of design and implementation quality	Section 22	Added
12 Reflection	A section entitled “Reflection” with a learning account for each team member	Section 23	Fordring drafted; Franklin entry requires confirmation
13 AI list	List of prompts and generative-AI platforms used	Section 24	Added; update with final prompt history
14 References	Academic and technical references	Section 16 and Section 25	Added

For final export, the report should retain sections 1–25 in order and include the linked SVG diagrams as figures or an appendix. The assignment page limits apply to the named subsections, not to the evidence appendix.

18. Software Lifecycle Discussion

The project adopted a **plan-driven lifecycle with iterative refinement**. The initial requirements, use cases, candidate objects, GUI prototypes, architecture sketches, and implementation plan were established before the main coding work. This was appropriate for an assessed design project because it made the expected deliverables and interfaces visible early and created a stable basis for traceability.

The process was not a rigid waterfall in practice. Requirements confirmation exposed ambiguity in the key use cases, particularly the guest redemption flow and manager operations. The team revisited the use cases, adjusted the design, and then implemented the clarified behavior. Tests and UML evidence were refined alongside the implementation. This hybrid approach retained the documentation and review benefits of a plan-driven process while allowing controlled iteration when requirements or implementation evidence changed.

The central tension was between **complexity management** and **agility**. Too little structure would have produced duplicated business rules, unclear ownership, and weak traceability; too much up-front design would have made it expensive to respond to discoveries. The project addressed this by fixing the core domain vocabulary and architectural boundaries early, then isolating variation behind interfaces and patterns: payment strategies, order states, mappers, event listeners, and the message producer/ consumer boundary. Small tests and explicit implementation-gap notes provide feedback without requiring a full redesign. This gives the team a controlled way

to add payment types, listeners, or earning rules while keeping the main redemption flow stable.

19. Project Plan Summary

The project plan divides work across requirements, architecture, GUI, coding, testing, documentation, and final review. The dependency order used by the project is:

```
Business scenario and use cases
        -> candidate objects and GUI prototypes
        -> architecture and UML
        -> implementation
        -> automated tests and traceability
        -> critique, reflection, AI list, and final references
```

The implementation evidence is organized around six use cases: UC1 login/account, UC2 points history, UC3 product redemption, UC4 points rules, UC5 manual point award, and UC6 product management. UC3 and UC6 are the two key use cases with detailed spreadsheet descriptions. The current schedule and named ownership remain in -v1.0-3-10-2026.xlsx; this README supplies the narrative that connects the plan to the delivered artifacts.

20. Quality Attributes and Quality Tactics

The project treats the following qualities as architectural concerns:

Quality attribute	Tactics and implementation evidence
Extensibility	Strategy interfaces for payment, state objects for order lifecycle, event listeners for reward distribution, mapper/service interfaces, and dependency injection
Security	Session authentication, role-based URL rules, BCrypt, generic login errors, explicit account-status checks, and guest ownership validation
Reliability	Transactional redemption, conditional balance/stock updates, persistent Artemis messaging, listener rollback/redelivery, and failure-visible implementation-gap documentation
Modifiability	Layered separation of controllers, services, mappers, and views; focused DTOs/VOs; constructor injection; isolated payment and messaging abstractions
Performance	Database-side conditional updates, filtered available-product queries, and asynchronous manual point adjustment
Testability	Constructor injection, service boundaries, mapper abstractions, Spring test slices, security test support, and a test pyramid with fast unit tests around the business core
Auditability	Signed points transactions, related order identifiers, timestamps, role checks, and explicit domain/application boundaries

The main trade-off is that the prototype favors local reproducibility over production scale: H2 and embedded Artemis reduce setup cost, while a production deployment would normally use a managed relational database, an external broker, CSRF protection, and external payment providers.

21. Transparency and Traceability

The source package responsibilities, class-level implementation, and test evidence are described in Sections 4, 5, and 12. The traceability chain is:

```
Assignment requirement
    -> use case / quality attribute
    -> UML or GUI artifact
    -> controller/service/mapper implementation
    -> automated test
    -> known gap or completion status
```

The current package-to-responsibility table is in Section 4, the endpoint-to-use-case mapping is in Section 8, and the test-to-behavior summary is in Section 12. A final submission should append a generated package/class/LOC table from the exact final commit, because LOC and class counts change as implementation and tests evolve. Authorship should be taken from Git history rather than inferred from file names.

22. Critique of Design and Implementation

The design is strong where the main business transaction crosses multiple concerns: authentication, payment validation, point deduction, stock deduction, order status, and audit history are separated into understandable boundaries. Strategy, State, Observer, Factory, Data Mapper, and message-driven processing are not merely named in the report; they correspond to identifiable code structures and diagrams. Conditional updates and transaction boundaries also address the most important correctness risk: concurrent redemptions.

The main weaknesses are completeness and production hardening. Points rules can be maintained but are not yet connected to an external hotel-order earning flow. Manual adjustments have no user-facing lifecycle or dead-letter administration, and the reason is not persisted. Guest product availability is filtered by the backend but is not represented as a complete availability state in the view model. Order cancellation and fulfilment states are modelled but not wired into workflows. Finally, the prototype disables CSRF and simulates cash/voucher payment; both choices are acceptable for a local academic demonstration but require replacement before production use.

Overall, the implementation is a credible and testable prototype with good separation of concerns. Its quality would improve most by closing the four documented gaps and by adding browser-level end-to-end tests for the complete guest and manager journeys.

23. Reflection

Fordring

This project taught me how Chain of Responsibility, Observer, Strategy, and State patterns are used in a real application rather than only in isolated examples. I also studied Spring Boot MVC, layered architecture, security, persistence, and asynchronous processing in more depth. During requirements confirmation, discussion of the key use cases revealed ambiguity and caused part of the work to be restarted, which affected the schedule. The team recovered by prioritizing the clarified use cases, working through the implementation and test evidence, and bringing the schedule back on track. The main lesson is that early use-case agreement is not administrative overhead: it is a technical dependency for architecture, implementation, and testing.

Franklin

What I learned

The most important thing I learned is the value of setting rules first. At the start, we wrote the architecture document, the coding standards (based on the Alibaba Java Development Manual), and the test strategy. When problems came up, we went back to these rules, found the cause, and improved the rules.

For example, the first version of selectAll() did not filter out items with no stock. We missed this while coding. Before writing the tests, we checked the code against the UC3 sequence diagram and found it. So the rules helped us find a mistake that the code alone did not show.

Project value

We worked in a “rules first” way: documents and standards came before the code, and the test strategy came before the tests.

What worked

Setting rules first for the “how” part, such as UML and the test strategy, gave us a clear base for later work.

What did not work

We drew the GUI prototypes for the “what” part too late, after the design phase. As a result, the team understood some requirements differently, such as what the “threshold” in the points rule meant. The lesson is to draw prototypes before the design, so everyone agrees on what we are building before we decide how.

24. Generative AI Use

Generative AI was used as an engineering assistant for repository inspection, explanation, documentation drafting, test-oriented reasoning, and consistency checking. It did not replace the team's responsibility for requirements decisions, code review, testing, or the accuracy of the final report.

The final submission should include the actual prompt history used by each team member, with the platform and approximate purpose. The current project-level record includes:

Platform	Purpose to record in the final prompt log
GitHub Copilot / Copilot SDK in VS Code	Repository analysis, implementation assistance, documentation, and test review
DeepSeek V4 Flash	Alternative explanations and review assistance, if used
Qwen3-Coder 30B	Code-oriented reasoning, if used
JoyAI Code 1.6	Code-oriented assistance, if used

Prompts should be listed as a short appendix or table; secrets, credentials, and unnecessary private data must not be included.

25. References and Submission Checklist

Formal reference list

1 Bass, Len, Paul Clements, and Rick Kazman (2013). *Software Architecture in

Practice* (3rd ed.). Addison-Wesley. Used for architectural drivers, quality attributes, tactics, and architecture evaluation.

1 Gamma, Erich, Richard Helm, Ralph Johnson, and John Vlissides (1994). *Design

Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley. Used for Strategy, State, Observer, Factory, and Chain of Responsibility patterns.

1 Fowler, Martin (2002). *Patterns of Enterprise Application Architecture*.

Addison-Wesley. Used for MVC, service-layer separation, Data Mapper, and enterprise application structuring.

1 Walls, Craig (2022). *Spring in Action* (6th ed.). Manning Publications. Used for

Spring Boot, Spring MVC, dependency injection, web security, and application configuration.

1 Martin, Robert C. (2002). *Agile Software Development: Principles, Patterns, and

Practices*. Prentice Hall. Used for the lifecycle discussion, iterative refinement, SOLID principles, and the balance between complexity management and agility.

The repository files, UML diagrams, use-case workbooks, and GitHub history are project evidence rather than bibliographic references. They are cited in the traceability matrix and supporting-artifact sections above.

Final export checklist

1 [ ] Confirm the two page-limited discussions meet their page limits.

1 [ ] Insert the two detailed key-use-case descriptions and the required diagrams.

1 [ ] Refresh package/class/LOC/authorship tables from the final commit.

1 [ ] Add the required version-control screenshots, if the marking format requires images.

1 [ ] Replace the Franklin reflection placeholder with the team member's own account.

1 [ ] Append the final AI prompt list and verify that it contains no secrets.

1 [ ] Run ./mvnw clean test and record the final test result and test count.

1 [ ] Review all “gap” labels so planned work is not presented as implemented behavior.

1 [ ] Export this README to PDF/HTML with SVG figures rendered and links preserved.